

Physicalized Model Weights - cycles 5-7

2026-05-13

Contents

Physicalized Model Weights - cycles 5-7	1
Abstract	1
Introduction	1
Methodology	2
Results	2
Discussion	6
Open Questions	7
Conclusions	8
References	8
Appendix: Implementation Details	8

Physicalized Model Weights - cycles 5-7

Abstract

Cycles 5-7 completed the first research arc on physicalizing model weights into hardware. The work moved from a validated hybrid safety-filter architecture into a concrete prototype, closed a verification gap around the hardware description language (HDL) evidence, and produced a final evidence-labeled synthesis.

The final conclusion is narrow. The completed work does not support permanently fixing full frontier large-language-model weights in hardware. It supports a smaller claim: physicalization is credible only for stable, bounded safety/filter submodels when they are placed behind programmable fallback, signed update, audit, health, drift, and rollback controls. This conclusion was validated by the cycle 7 audit after one manifest-completeness defect was fixed.

Introduction

The directive asks whether useful portions of neural-network inference can be “physicalized” into hardware instead of repeatedly executed as software kernels over programmable processors and memory hierarchies. In this report, “physicalized” means that some part of inference is moved from general programmable execution into a hardware structure whose storage, topology, movement, or compute behavior encodes the workload.

Cycles 2-4 had already narrowed the research direction. They rejected broad full-model physicalization, ranked fixed or semi-fixed safety/filter classifier submodels as the strongest target, and proposed a host-controlled, RISC-V-compatible hybrid architecture. Cycles 5-7 focused on turning that architecture into a small inspectable prototype and then consolidating the first arc into a final synthesis.

A labeling note: the source sessions sometimes use “cycle 1” inside worker documents because they describe local milestone passes. This report follows the orchestrator’s requested global range, cycles 5-7.

Methodology

The cycle 5 researcher session 85d373ea-e7a4-42a9-a18f-b1fdb143e3f3 opened M-PROTO-1, the prototype milestone. The brief asked for a tiny fixed safety/filter classifier, a Python golden model, route and fallback outputs, a small HDL core, Yosys and Graphviz checks, and tests.

The cycle 6 researcher session e4b61515-6f1c-4efd-b914-60095f678539 kept the scope narrow. It did not ask for a larger accelerator model. It asked only to close the verification gap left by cycle 5: compiled Verilator simulation had been requested, but the local environment lacked the build tools needed to run it.

The cycle 7 researcher session cc15866e-6479-45a0-bdf4-a86cb3aec9be opened M-FINAL-1, the final synthesis milestone. Its instruction was to integrate the validated milestones, separate evidence levels, preserve the null hypothesis, and avoid adding new architecture claims unless the synthesis exposed a missing supported claim.

Results

Cycle 5: Prototype Safety-Filter Fast Path

Cycle 5 built the first concrete prototype of the validated safety/filter target. The worker session 79f5740b-c17d-427a-8f21-5a5186eeb8c3 produced:

- physicalized-weights/scripts/prototype_safety_filter.py
- physicalized-weights/hdl/safety_filter_core.sv
- physicalized-weights/hdl/safety_filter_core_tb.cpp
- physicalized-weights/hdl/safety_filter_core.js
- physicalized-weights/hdl/run_yosys_eval.py
- physicalized-weights/tests/test_prototype_safety_filter.py
- physicalized-weights/docs/prototype_safety_filter.md

The prototype classifier used 8 signed int8 features, fixed signed int8 weights, a bias of -10, a threshold of 64, and confidence equal to the distance from the threshold. Its fixed weights were:

```
[12, -7, 5, 9, -11, 4, 6, -3]
```

The classifier decision was `block` when the score was greater than or equal to the threshold and `allow` otherwise. The routing policy then decided whether the fixed classifier output could be used. Low confidence, stale policy version, failed health, drift alarm, host-forced fallback, audit failure, classifier unavailability, or invalid output routed away from the physicalized fast path.

The generated prototype summary recorded 16 cases:

Route	Count	Fraction
physicalized_fast_path	6	0.375
programmable_fallback	8	0.500
fail_safe	2	0.125

The baseline comparison used normalized cost units and kept feature extraction, register/control, audit, fallback dispatch, and fail-safe handling in the accounting:



Figure 1: Route distribution for the prototype safety-filter classifier, separating physicalized fast path, programmable fallback, and fail-safe outcomes across nominal and edge-case vectors.

Baseline	Cases	Fast Path	Fallback	Fail-Safe	Modeled Cost Units
software_	16	0	16	0	400.0
optimized					
programmable_	16	0	16	0	368.0
accelerator					
hybrid_	16	6	8	2	162.0
physicalized					
hybrid_	0	0	0	0	0.0
request_					
volume_					
zero					

The worker interpreted the result as support for the architecture boundary, not proof of a hardware win. The positive result was that fixed computation could stay small and deterministic. The negative result was that the meaningful system behavior lived around the dot product: feature extraction, policy, audit, fallback, and fail-safe routing.

The cycle 5 audit session 7c68e65c-452c-4f42-b100-5698ab5d04cf found no critical defects. It confirmed that Python prototype generation, Yosys evaluation, Verilator lint, Yosys synthesis, Graphviz rendering, and stdlib tests passed. It also confirmed the main limitation: compiled Verilator simulation did not run because make was unavailable. The milestone correctly remained in-progress.

Cycle 6: Prototype Verification Closure

Cycle 6 closed the Verilator gap without expanding the prototype. The worker session 5540a86d-4a88-439a-8044-e24f6ee515b5 created:

- physicalized-weights/docs/prototype_verification_closure.md
- physicalized-weights/scripts/verify_prototype_closure.py
- physicalized-weights/tests/test_prototype_verification_closure.py
- physicalized-weights/data/prototype_verification_closure.json
- physicalized-weights/data/prototype_equivalence_matrix.csv
- physicalized-weights/data/prototype_equivalence_matrix.png

The closure recorded that Verilator itself existed, but make and a C++ compiler did not. The compiled simulation was therefore marked `blocked_make_unavailable`, not passed.

The amended evidence contract accepted Verilator lint plus Yosys evaluation, Yosys synthesis, Graphviz artifacts, Python golden-vector agreement, and artifact hash binding as sufficient for this specific core. The rationale was narrow: the HDL core is pure combinational logic with no clock, reset, memory, handshake timing, or mutable policy state. It contains only fixed localparam weights, a signed dot product, a threshold comparison, and margin/confidence outputs.



Figure 2: Equivalence matrix showing agreement among Python golden model, Yosys HDL evaluation, lint/synthesis checks, and generated artifacts.

The closure JSON recorded:

Field	Value
closure_status	validated
evidence_contract	amended_lint_yosys_eval_synthesis
compiled_simulation_status	blocked_make_unavailable
yosys_eval_matches_python	true
verilator_lint_passed	true
yosys_synthesis_passed	true
graphviz_artifacts_present	true
structural_artifacts_fresh	true

The Yosys/Graphviz structural path produced the safety-filter netlist diagram:

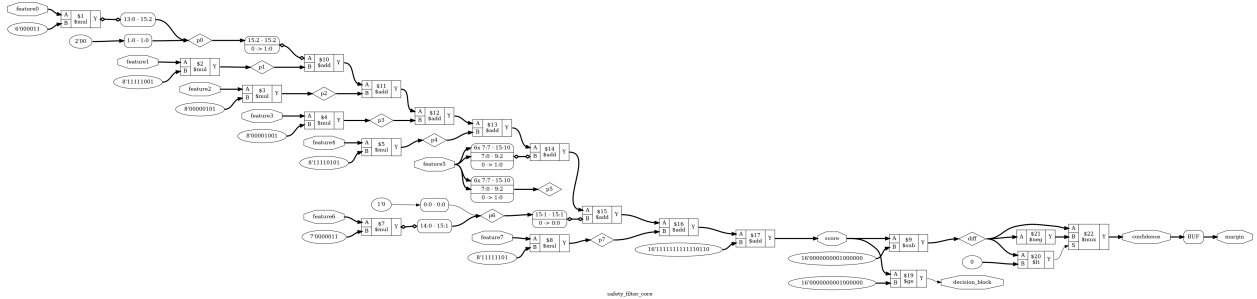


Figure 3: Yosys/Graphviz netlist rendering for the fixed safety-filter HDL core.

The cycle 6 audit session f98be601-593a-47c5-99fa-d67d098ef1ff found one moderate issue and fixed it. The worker closure had recorded hashes for generated artifacts, but did not prove those artifacts were fresh relative to the current HDL and Yosys input scripts. The auditor added structural_artifacts_fresh enforcement to verify_prototype_closure.py and updated test_prototype_verification_closure.py.

After rerunning the checks, the auditor validated M-PROTO-1 with high confidence and appended an auditor ledger event. The reopen rule remained explicit: reopen the milestone if compiled Verilator later runs and disagrees, Python/Yosys rows diverge, the HDL hash changes without regenerated evidence, or the HDL gains sequential state, memories, handshake timing, or mutable policy logic.

Cycle 7: Final Evidence-Labeled Synthesis

Cycle 7 produced the final synthesis package for the first research arc. The worker session f2501ef1-a71e-44f9-bf6d-0c5708d46f13 created:

- physicalized-weights/docs/final_synthesis.md
- physicalized-weights/docs/reproducibility.md
- physicalized-weights/scripts/build_final_synthesis.py
- physicalized-weights/tests/test_final_synthesis.py
- physicalized-weights/data/evidence_manifest.csv
- physicalized-weights/data/evidence_manifest.json
- physicalized-weights/data/final_synthesis_summary.json
- physicalized-weights/data/final_evidence_map.png

The worker’s final synthesis stated the central answer directly: full frontier-model fixed-weight physicalization is unsupported, while narrow physicalization of a stable safety/filter

classifier remains credible only behind programmable fallback, update, audit, and rollback controls.

The summary JSON recorded the validated milestones:

Milestone	Meaning
M-TAX-1	taxonomy and null hypothesis
M-MODEL-1	break-even model
M-BASE-1	software/runtime and programmable baselines
M-TARGET-1	ranked target selection
M-ARCH-1	hybrid architecture
M-PROTO-1	prototype and verification closure
M-FINAL-1	final synthesis

The first final-synthesis build produced 23 manifest artifacts. The cycle 7 audit session d1a61277-63fb-47c8-8278-79f669aadd85 found one moderate defect: the final report used sourced and speculative claim labels, but the generated manifest and summary had zero rows for those categories. The auditor fixed `build_final_synthesis.py` and `test_final_synthesis.py` so all six evidence categories had nonzero representation.

After the fix, the final evidence manifest contained 25 artifacts, and the evidence type counts were:

Evidence Type	Count
sourced	1
modeled	5
simulated	4
synthesized	4
inferred	10
speculative	1

The final audit validated M-FINAL-1. It confirmed that the synthesis answered the central question, preserved the software/runtime null hypothesis, rejected broad fixed frontier-model physicalization, preserved the narrow safety/filter conclusion, included falsification criteria, included reproducibility commands, and documented the M-PROTO-1 limitations and reopen rules.

Discussion

Cycles 5-7 converted the earlier architecture into a research-grade first arc. The key result is not that a tiny fixed dot product is valuable by itself. The key result is the separation of concerns:

- fixed hardware computes only a bounded, deterministic classifier score, decision, and confidence;
- programmable control owns policy versioning, health checks, drift checks, audit logging, fallback, rollback, and final routing;
- the system treats low-confidence or invalid conditions conservatively by routing away from the physicalized output.



Figure 4: Evidence map linking validated milestones, generated artifacts, and final claims by evidence type.

This separation supports the narrow physicalization claim while reinforcing the null hypothesis. Most practical system complexity still lives in software, runtime control, and programmable fallback. A design that hides that complexity inside fixed hardware would stop matching the validated architecture.

The RISC-V path remains credible as an open control-plane and integration substrate because RISC-V is a free and open instruction set with public specifications [1], [2]. The report does not treat RISC-V as automatically optimal. It treats it as an inspectable way to study a memory-mapped or host-controlled accelerator boundary.

Analog and other device-specific physicalization paths remain speculative in this arc. IBM’s analog in-memory framing supports the general motivation of reducing weight-movement costs [6], but the work here does not provide device-level calibration, drift, conversion overhead, yield, repair, or manufacturing evidence for analog hardware.

Open Questions

The next cycles should begin from the falsification roadmap rather than broadening the claim.

The main open work is to replace diagnostic cases and normalized cost units with workload traces and calibrated costs. The needed measurements include safety/filter invocation volume, fallback rate, policy update cadence, audit write cost, feature extraction cost, accelerator dispatch cost, memory-access cost, utilization, and package/update economics.

Compiled Verilator simulation remains a future superseding check. It did not run locally because `make` and a C++ compiler were unavailable. If those tools become available, the compiled simulator should be compared against `physicalized-weights/data/hdl_sim_results.csv`; disagreement reopens M-PROTO-1.

The architecture should also be reopened if the HDL core grows beyond the validated bound-

ary. Sequential state, memories, handshake timing, data-dependent control flow, or mutable policy logic would require a stronger evidence contract than the one accepted for the pure combinational core.

Conclusions

Cycles 5-7 completed and validated the first research arc.

The supported answer is conditional: physicalized weights are not a credible broad replacement for programmable inference on the evidence gathered so far. They remain credible only for a stable, bounded, high-reuse safety/filter submodel that is small enough to isolate and conservative enough to route around. The decisive condition is still the amortization mechanism established earlier:

```
N * (C_prog - C_phys) > C_fixed + C_update + C_yield + C_integration
```

When request volume is low, update cadence is fast, fallback rate is high, feature extraction and audit costs dominate, or optimized software/runtime baselines close the memory-movement gap, physicalization loses. When reuse is high, updates are slow, behavior is bounded, and programmable fallback prevents silent failure, the narrow safety/filter fast path remains worth studying.

References

- [1] RISC-V International, “RISC-V FAQ,” RISC-V International. <https://riscv.org/about/faq/>
- [2] RISC-V International, “RISC-V Technical Specifications,” RISC-V International. <https://docs.riscv.org/refe>
- [6] IBM Research, “How can analog in-memory computing power transformer models?,” IBM Research. <https://research.ibm.com/blog/how-can-analog-in-memory-computing-power-transformer-models>

Appendix: Implementation Details

Code Organization

The workspace snapshot is recorded in MANIFEST.md. During this reporter pass, MANIFEST.md was replaced with a current snapshot through M-FINAL-1.

Authored research scripts now total 7 files and 2,215 lines:

File	Lines	Purpose
physicalized-weights/scripts/breakeven_model.py	384	break-even sweep
physicalized-weights/scripts/target_scoring.py	283	target and anti-target scoring
physicalized-weights/scripts/fallback_policy_sim.py	342	fallback policy simulation
physicalized-weights/scripts/prototype_safety_filter.py	392	fixed classifier prototype

File	Lines	Purpose
physicalized-weights/scripts/verify_prototype_closure.py	377	prototype closure checks
physicalized-weights/scripts/build_final_synthesis.py	409	evidence manifest, summary, and evidence map builder
physicalized-weights/scripts/symbolic_breakeven.wls	28	symbolic break-even derivation

Authored tests now total 6 files and 726 lines:

File	Lines
physicalized-weights/tests/test_breakeven_model.py	108
physicalized-weights/tests/test_target_scoring.py	133
physicalized-weights/tests/test_fallback_policy_sim.py	117
physicalized-weights/tests/test_prototype_safety_filter.py	128
physicalized-weights/tests/test_prototype_verification_closure.py	119
physicalized-weights/tests/test_final_synthesis.py	121

HDL and HDL-support files total 4 files and 241 lines:

File	Lines	Purpose
physicalized-weights/hdl/safety_filter_core.sv	45	fixed dot-product HDL core
physicalized-weights/hdl/safety_filter_core_tb.cpp	72	future compiled-Verilator testbench
physicalized-weights/hdl/safety_filter_core.yo	11	Yosys script
physicalized-weights/hdl/run_yosys_eval.py	113	Yosys evaluation harness

Research docs and diagram sources total 8 files and 673 lines.

Generated Data and Figures

The cycle 5 prototype generated:

- physicalized-weights/data/prototype_vectors.csv
- physicalized-weights/data/prototype_route_results.csv
- physicalized-weights/data/prototype_baseline_comparison.csv
- physicalized-weights/data/prototype_summary.json
- physicalized-weights/data/prototype_route_distribution.png

The cycle 6 closure generated:

- physicalized-weights/data/hdl_sim_results.csv
- physicalized-weights/data/verilator_safety_filter.log
- physicalized-weights/data/yosys_safety_filter.log
- physicalized-weights/data/safety_filter_core_netlist.dot
- physicalized-weights/data/safety_filter_core_netlist.png
- physicalized-weights/data/prototype_verification_closure.json
- physicalized-weights/data/prototype_equivalence_matrix.csv
- physicalized-weights/data/prototype_equivalence_matrix.png

The cycle 7 synthesis generated:

- physicalized-weights/data/evidence_manifest.csv
- physicalized-weights/data/evidence_manifest.json
- physicalized-weights/data/final_synthesis_summary.json
- physicalized-weights/data/final_evidence_map.png

Figure dimensions recorded during reporting:

Figure	Dimensions
prototype_route_distribution.png	720 x 420
prototype_equivalence_matrix.png	552 x 356
safety_filter_core_netlist.png	4488 x 1071
final_evidence_map.png	980 x 520
hybrid_safety_filter_arch.png	1737 x 623
breakeven_update_volume.png	402 x 294
target_score_heatmap.png	696 x 400

Test Results

The cycle 5 worker reported these commands passing:

```
python3 physicalized-weights/scripts/prototype_safety_filter.py
python3 physicalized-weights/hdl/run_yosys_eval.py
yosys -s physicalized-weights/hdl/safety_filter_core.ys > physicalized-weights/data/yosys_safety_filter.log 2>&1
dot -Tpng physicalized-weights/data/safety_filter_core_netlist.dot -o
↳ physicalized-weights/data/safety_filter_core_netlist.png
python3 physicalized-weights/tests/test_prototype_safety_filter.py
python3 physicalized-weights/tests/test_fallback_policy_sim.py
python3 physicalized-weights/tests/test_target_scoring.py
python3 physicalized-weights/tests/test_breakeven_model.py
python3 -m long_exposure.tools.promise_check .
```

The cycle 6 auditor reported these checks passing after the freshness fix:

```
python3 physicalized-weights/hdl/run_yosys_eval.py
python3 physicalized-weights/scripts/verify_prototype_closure.py
```

```
python3 physicalized-weights/tests/test_prototype_verification_closure.py
python3 physicalized-weights/tests/test_prototype_safety_filter.py
```

The cycle 7 auditor reported these checks passing after the evidence-category fix:

```
python3 physicalized-weights/scripts/build_final_synthesis.py
python3 physicalized-weights/tests/test_final_synthesis.py
python3 physicalized-weights/tests/test_breakeven_model.py
python3 physicalized-weights/tests/test_target_scoring.py
python3 physicalized-weights/tests/test_fallback_policy_sim.py
python3 physicalized-weights/tests/test_prototype_safety_filter.py
python3 physicalized-weights/tests/test_prototype_verification_closure.py
```

pytest remains unavailable in the current environment. Tests were run as stdlib Python harnesses.

Session References

	Cycle	Role	Session ID	Contribution
	5	researcher	85d373ea-e7a4-42a9-a18f-b1fdb143e3f3	opened M-PROTO-1 prototype brief
	5	worker	79f5740b-c17d-427a-8f21-5a5186eeb8c3	built Python prototype, HDL core, Yosys/Graphviz evidence, and prototype tests
	5	auditor	7c68e65c-452c-4f42-b100-5698ab5d04cf	validated available prototype evidence and kept milestone in progress due to missing compiled Verilator
	6	researcher	e4b61515-6f1c-4efd-b914-60095f678539	scoped narrow verification closure
	6	worker	5540a86d-4a88-439a-8044-e24f6ee515b5	created closure artifact, closure script, closure tests, and amended evidence contract
	6	auditor	f98be601-593a-47c5-99fa-d67d098ef1ff	fixed structural-artifact freshness gap and validated M-PROTO-1
	7	researcher	cc15866e-6479-45a0-bdf4-a86cb3aec9be	scoped M-FINAL-1 final synthesis
	7	worker	f2501ef1-a71e-44f9-bf6d-0c5708d46f13	built final synthesis, reproducibility doc, manifest builder, tests, and evidence map

Cycle	Role	Session ID	Contribution
7	auditor	d1a61277-63fb-47c8-8278-79f669aadd85	fixed evidence-category manifest gap and validated M-FINAL-1

Cross-Reference Map

The current cross-reference chain is:

```
directive
-> plan_of_record.md
-> promise_ledger.jsonl
-> taxonomy_and_null.md
-> breakeven_model.py and symbolic_breakeven.wls
-> target_scoring.py and target_ranking.md
-> hybrid_safety_filter_architecture.md
-> fallback_policy_sim.py
-> prototype_safety_filter.py
-> safety_filter_core.sv
-> run_yosys_eval.py and safety_filter_core.py
-> verify_prototype_closure.py
-> prototype_verification_closure.json
-> final_synthesis.md
-> build_final_synthesis.py
-> evidence_manifest.json and final_synthesis_summary.json
-> test_final_synthesis.py
```

The ledger now contains 15 events through auditor validation of M-FINAL-1. The remaining warnings reported by validators are minor or expected: pre-existing orphan cycle report files and root prompt/log warnings.